

Kanban Task Management System

ReactJS + NestJS + (RDBMS/NoSQL) + Swagger/OpenAPI

We need to build a Trello-style Kanban task management application. Users organize work into boards, columns and cards, drag cards between columns to change status, assign work to teammates and track activity. The project is intentionally simple in UI but requires clean service architecture, drag-and-drop persistence and role-based collaboration.

Area	Requirement
Frontend	ReactJS, React Router, React Query, Material UI
Backend	NestJS, REST APIs, DTO validation, JWT authentication
Database	ANY (RDBMS / NoSQL)
API Documentation	Swagger / OpenAPI — Mandatory
Architecture	Controller → Service → Repository/Model → DB
Suggested Duration	3-4 Days
Version Control	GitHub

1. Project Objective

Build a production-style task management platform where users create boards, invite members, create columns and cards, move cards through workflow stages via drag-and-drop, add comments and view an activity log.

Learning Objectives

- React component architecture and drag-and-drop with a library like dnd-kit.
- NestJS modules, controllers, services and DTO validation.
- JWT authentication and role-based board membership.
- Data modelling for hierarchical entities (Board → Column → Card).
- Optimistic UI updates with React Query.
- Position-based ordering algorithm for drag-and-drop.
- Swagger / OpenAPI documentation.

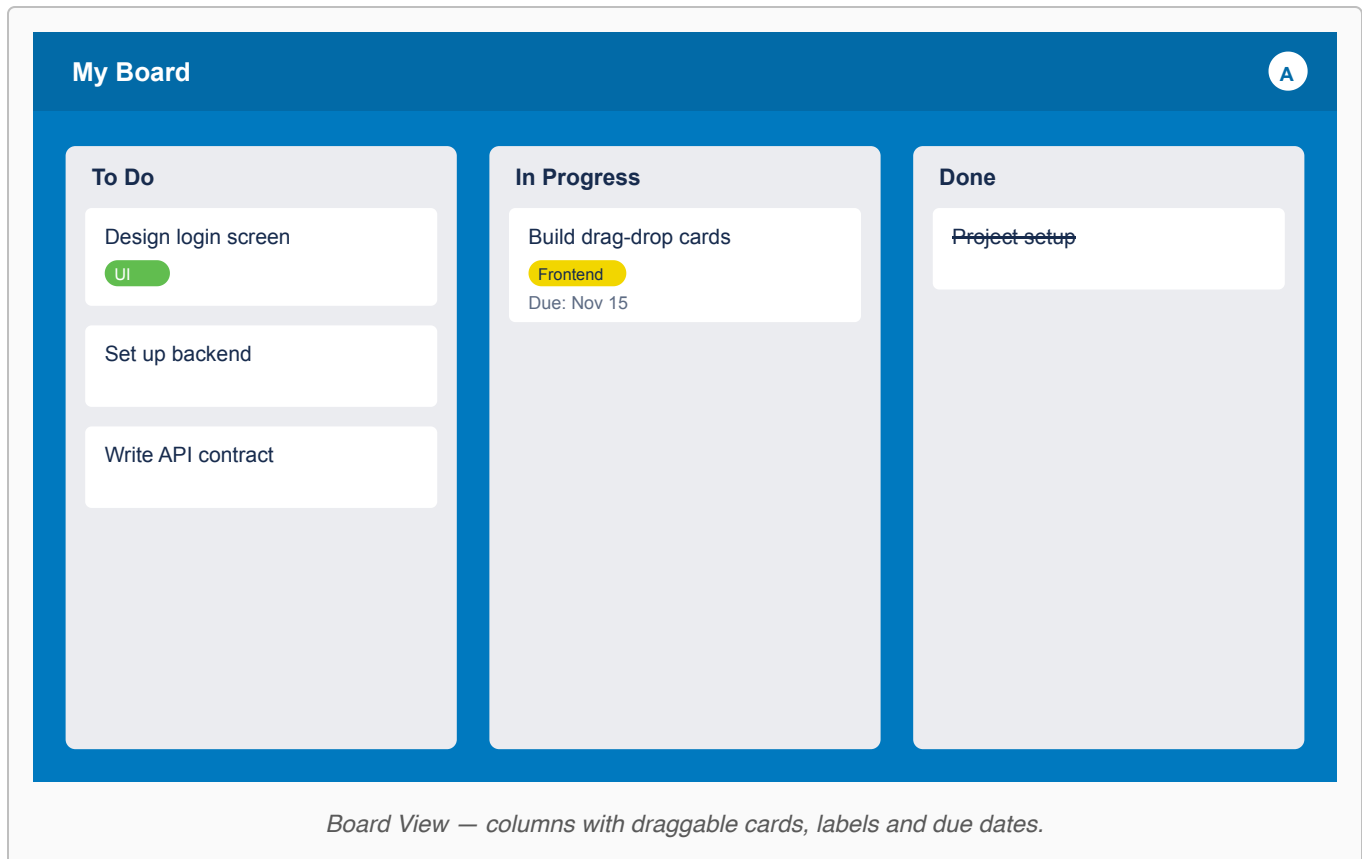
2. User Roles

Role	Responsibilities
------	------------------

Board Owner	Create/delete board, invite members, change member roles, all card actions.
Member	Create/edit/move/delete cards and columns, comment on cards.
Viewer	Read-only access to board, cards and comments.

3. Visual Reference

Use the embedded mockup as visual and information-architecture reference. Styling may be improved while preserving the required functionality.



4. Core Workflow

When a user moves a card across columns, the backend performs the following operations in order:

1. Validate request and user's board membership.
2. Validate target column belongs to the same board.
3. Compute new position value between neighbouring cards.
4. Update card column_id and position atomically.
5. Create an activity log entry (moved from X to Y).
6. Emit a notification to card watchers.

5. Business Rules

- A user must be a member of the board to view or modify it.
- Only Owner or Member can create / edit / delete cards.
- Viewer role is read-only and cannot mutate anything.
- Deleting a board requires confirmation and cascades to columns and cards.
- Card position must remain unique per column.
- An archived card cannot be moved or edited until restored.
- Every card change (create / move / edit / delete) creates an activity log entry.

6. Data Model

The data model may be built in a relational database or a NoSQL store (choice of the developer). The main entities that must exist:

```
Board      { id, ownerId, title, description, isArchived, createdAt, updatedAt }
BoardMember { id, boardId, userId, role }
Column     { id, boardId, title, position }
Card       { id, columnId, title, description, dueDate, labels, position, assignee }
Comment    { id, cardId, userId, text, createdAt }
Activity   { id, boardId, cardId?, userId, action, meta, createdAt }
User       { id, name, email, passwordHash }
```

7. REST API Requirements

API endpoints must be meaningful, RESTful and role-protected. At minimum the following capabilities are required:

- Auth: signup, login, refresh, get current user.
- Boards CRUD, invite/remove members, change member role.
- Columns CRUD with reorder endpoint.
- Cards CRUD with move endpoint (target column + position).
- Comments on cards.
- Activity feed per board and per card.

8. Swagger / OpenAPI — Mandatory

Every API endpoint must be documented in Swagger. The Swagger UI should be exposed at a route such as `/api/docs`.

- JWT Bearer authentication.
- Request DTO schemas with required/optional fields.
- Response schemas.

- HTTP status codes and error responses.
- Endpoint descriptions and examples.
- Query parameters for pagination, filtering, sorting and search.
- Role/authorization requirements where applicable.

9. ReactJS Screens

Screen	Requirements
Login / Signup	JWT login, redirect to boards list.
Board List	List of user's boards, create board, delete with confirm.
Board Detail	Columns and cards, drag-drop, add column, add card.
Card Detail (drawer)	Edit title/description/due/labels/assignees, comments, activity.
Members	Invite by email, change role, remove member.
Activity	Full board activity log with filters.

10. Search, Filtering and Pagination

The main list API should support: `page` , `limit` , `search` , `label` , `assignee` , `isArchived` , `sortBy` , `sortOrder` .

11. Reporting / Dashboard

- Cards per column (WIP counts).
- Cards completed this week / this month.
- Overdue cards.
- Cards per assignee.
- Board activity per day (last 30 days).

12. Advanced Logic — Optional

- Real-time updates via WebSockets.
- Card checklists (sub-tasks).
- File attachments on cards.
- Card templates.
- Board sharing via public read-only link.
- Dark mode.

13. Definition of Done

- All required screens are functional.
- APIs are protected according to user roles.
- Swagger documentation is complete and usable.
- Indexes are added for important query fields.
- Pagination/filtering works server-side.
- Business rules are implemented in the service layer, not controllers.
- Activity/audit history is maintained where applicable.
- Core business logic is covered by tests.
- README contains setup instructions, environment variables and Swagger URL.

Final Expected Outcome

You should be able to build and explain a full-stack task management app with clean NestJS service separation, drag-and-drop persistence, role-based board membership, comment threads, activity logs and complete Swagger documentation. The UI stays simple but the ordering, membership and activity logic must be correct.